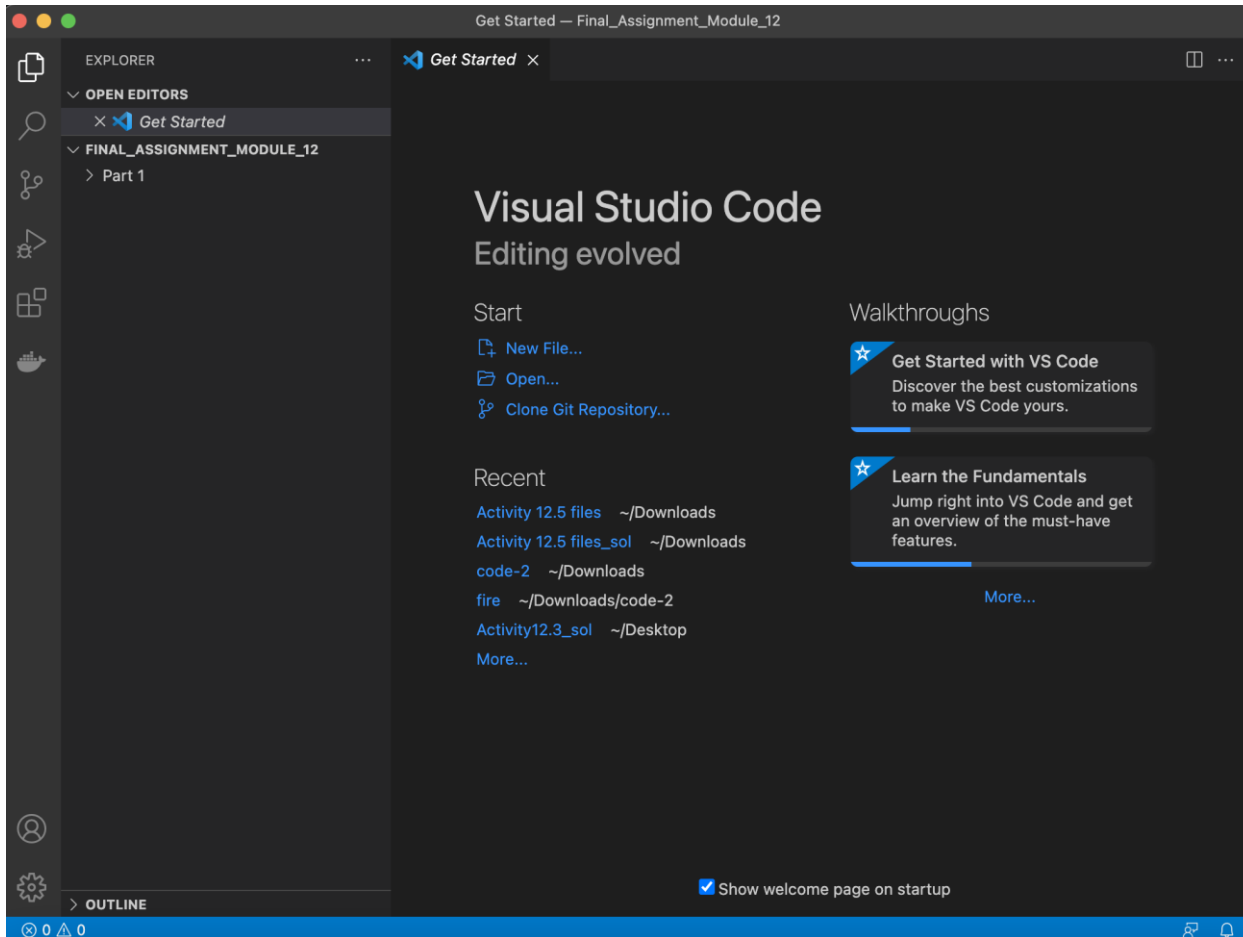
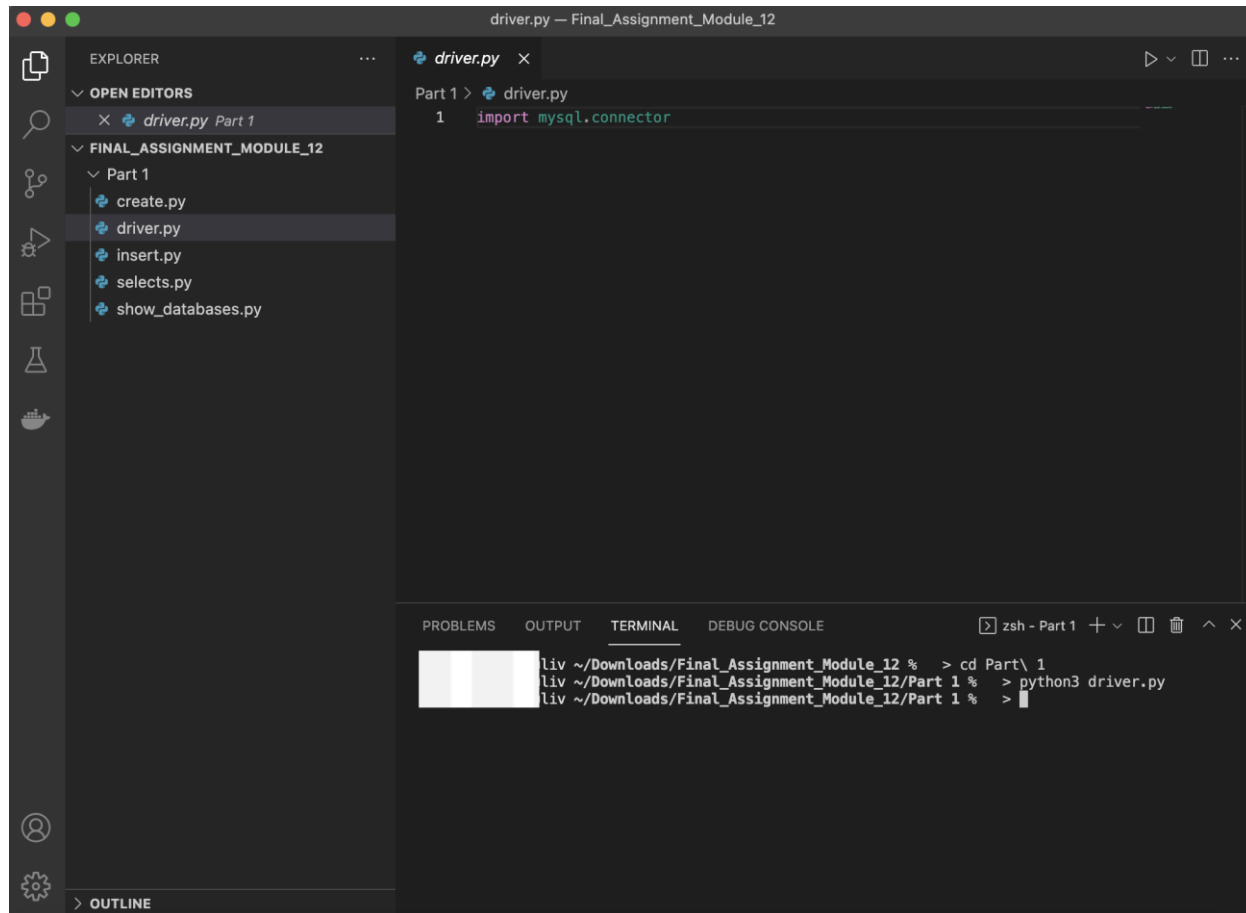


Final Assignment 12.1: Types of Databases and Database Containerization Solution

1)



Part 2a)



driver.py — Final_Assignment_Module_12

EXPLORER

OPEN EDITORS

driver.py Part 1

FINAL_ASSIGNMENT_MODULE_12

Part 1

- create.py
- driver.py
- insert.py
- selects.py
- show_databases.py

driver.py

```
1 import mysql.connector
```

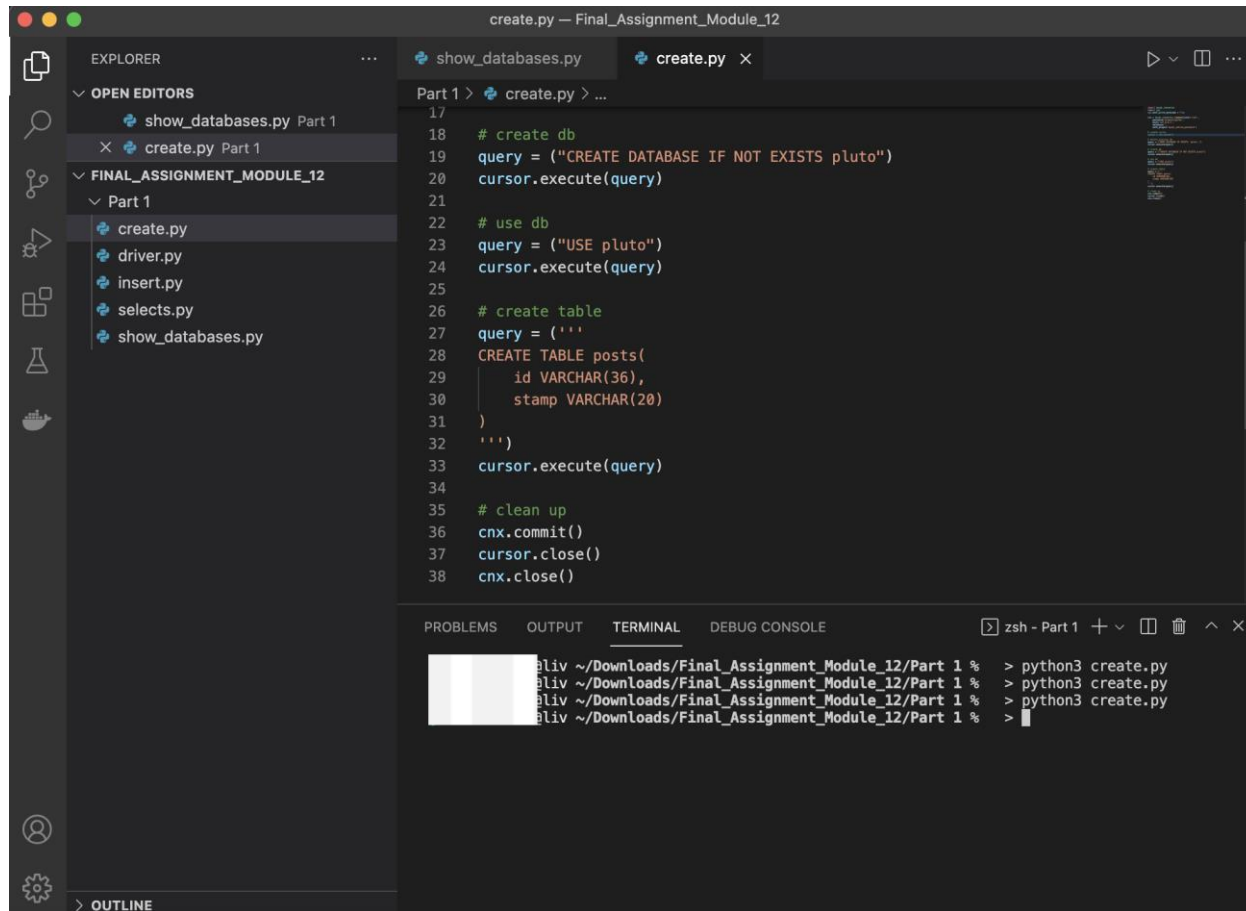
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

zsh - Part 1

```
liv ~/Downloads/Final_Assignment_Module_12 % > cd Part\ 1
liv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 driver.py
liv ~/Downloads/Final_Assignment_Module_12/Part 1 % >
```

OUTLINE

Part 2b)

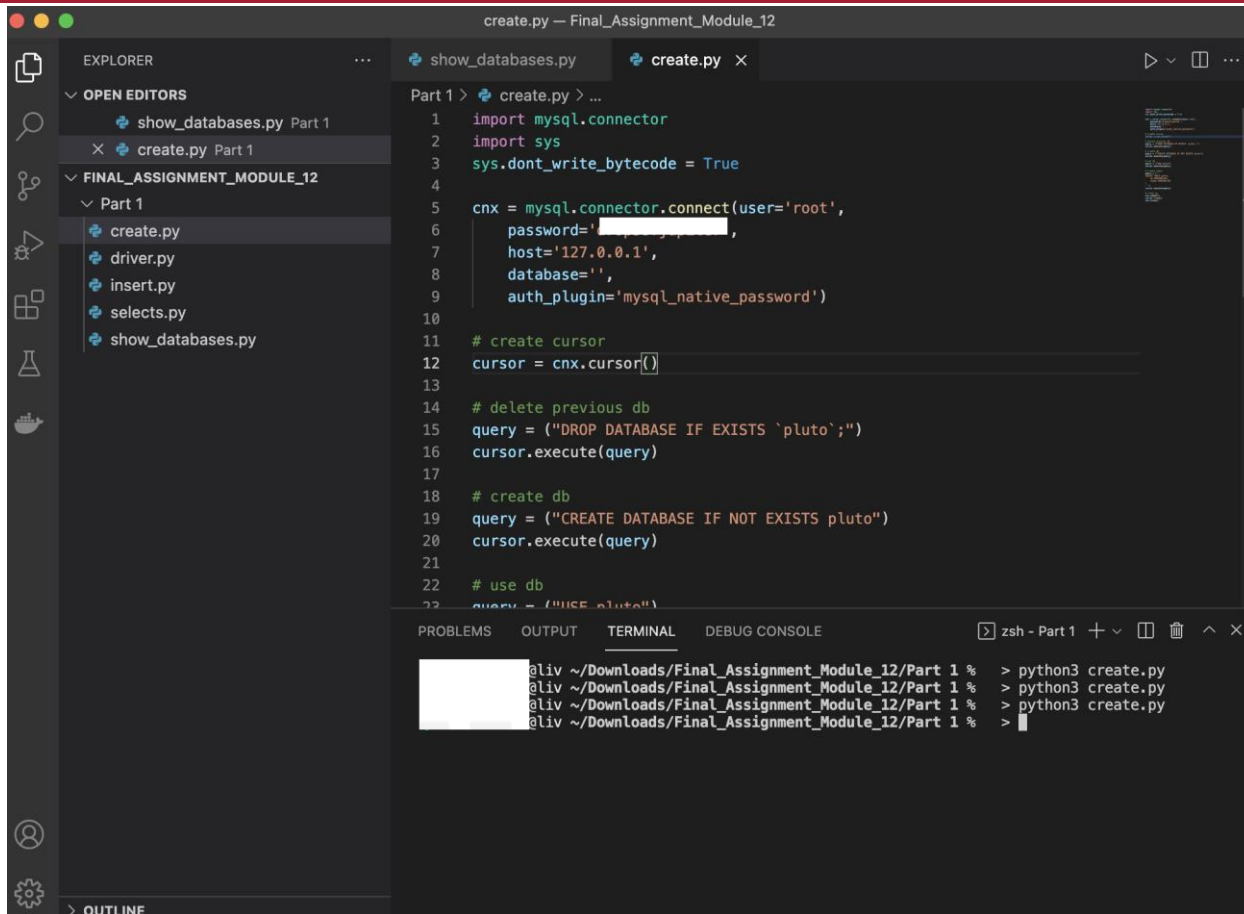


The screenshot displays a Visual Studio Code editor window titled "create.py — Final_Assignment_Module_12". The Explorer sidebar on the left shows the project structure with files: show_databases.py, create.py (Part 1), driver.py, insert.py, selects.py, and show_databases.py. The main editor area shows the code for create.py, which includes SQL queries to create a database, use it, create a table, and clean up. The terminal at the bottom shows the command "python3 create.py" being executed multiple times.

```
17
18 # create db
19 query = ("CREATE DATABASE IF NOT EXISTS pluto")
20 cursor.execute(query)
21
22 # use db
23 query = ("USE pluto")
24 cursor.execute(query)
25
26 # create table
27 query = ('''
28 CREATE TABLE posts(
29     id VARCHAR(36),
30     stamp VARCHAR(20)
31 )
32 ''')
33 cursor.execute(query)
34
35 # clean up
36 cnx.commit()
37 cursor.close()
38 cnx.close()
```

Terminal output:

```
zsh - Part 1
~ /Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
~ /Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
~ /Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
~ /Downloads/Final_Assignment_Module_12/Part 1 % >
```



create.py — Final_Assignment_Module_12

EXPLORER

- OPEN EDITORS
 - show_databases.py Part 1
 - create.py Part 1
- FINAL_ASSIGNMENT_MODULE_12
 - Part 1
 - create.py
 - driver.py
 - insert.py
 - selects.py
 - show_databases.py

Part 1 > create.py > ...

```

1 import mysql.connector
2 import sys
3 sys.dont_write_bytecode = True
4
5 cnx = mysql.connector.connect(user='root',
6                               password='[REDACTED]',
7                               host='127.0.0.1',
8                               database='',
9                               auth_plugin='mysql_native_password')
10
11 # create cursor
12 cursor = cnx.cursor()
13
14 # delete previous db
15 query = ("DROP DATABASE IF EXISTS `pluto`;")
16 cursor.execute(query)
17
18 # create db
19 query = ("CREATE DATABASE IF NOT EXISTS pluto")
20 cursor.execute(query)
21
22 # use db
23 query = ("USE pluto")

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

zsh - Part 1

```

@liv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
@liv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
@liv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
@liv ~/Downloads/Final_Assignment_Module_12/Part 1 % >

```

> OUTLINE

Part 2c)

The screenshot shows a VS Code editor window titled "selects.py — Final_Assignment_Module_12". The Explorer sidebar on the left shows the project structure with "Part 1" containing several Python files. The main editor displays the code for "selects.py", which imports MySQL connector, datetime, and uuid modules, connects to a MySQL database, and executes a query to select all data from a table named "posts". The terminal at the bottom shows the execution of the script, which prints three rows of data.

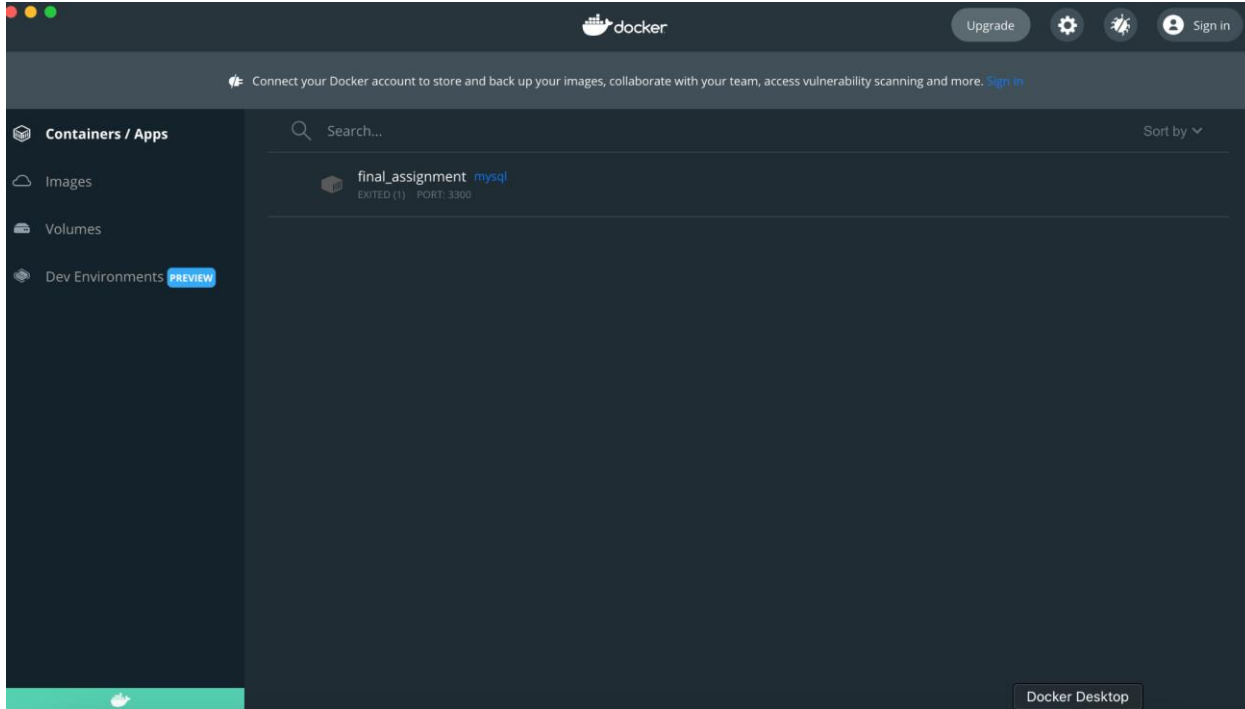
```
selects.py — Final_Assignment_Module_12
EXPLORER
  OPEN EDITORS
    show_databases.py Part 1
    create.py Part 1
    insert.py Part 1
    selects.py Part 1
  FINAL_ASSIGNMENT_MODULE_12
    Part 1
      create.py
      driver.py
      insert.py
      selects.py
      show_databases.py

Part 1 > selects.py > ...
1 import mysql.connector
2 from datetime import datetime
3 import uuid
4 import sys
5 sys.dont_write_bytecode = True
6
7 cnx = mysql.connector.connect((user='root',
8     password=
9     host='127.0.0.1',
10    database='pluto',
11    auth_plugin='mysql_native_password'])
12
13 # create cursor
14 cursor = cnx.cursor()
15
16 # insert
17 query = ("SELECT * FROM posts")
18 cursor.execute(query)
19
20 for row in cursor.fetchall():
21     print(row)
22
23 # clean up
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE zsh - Part 1

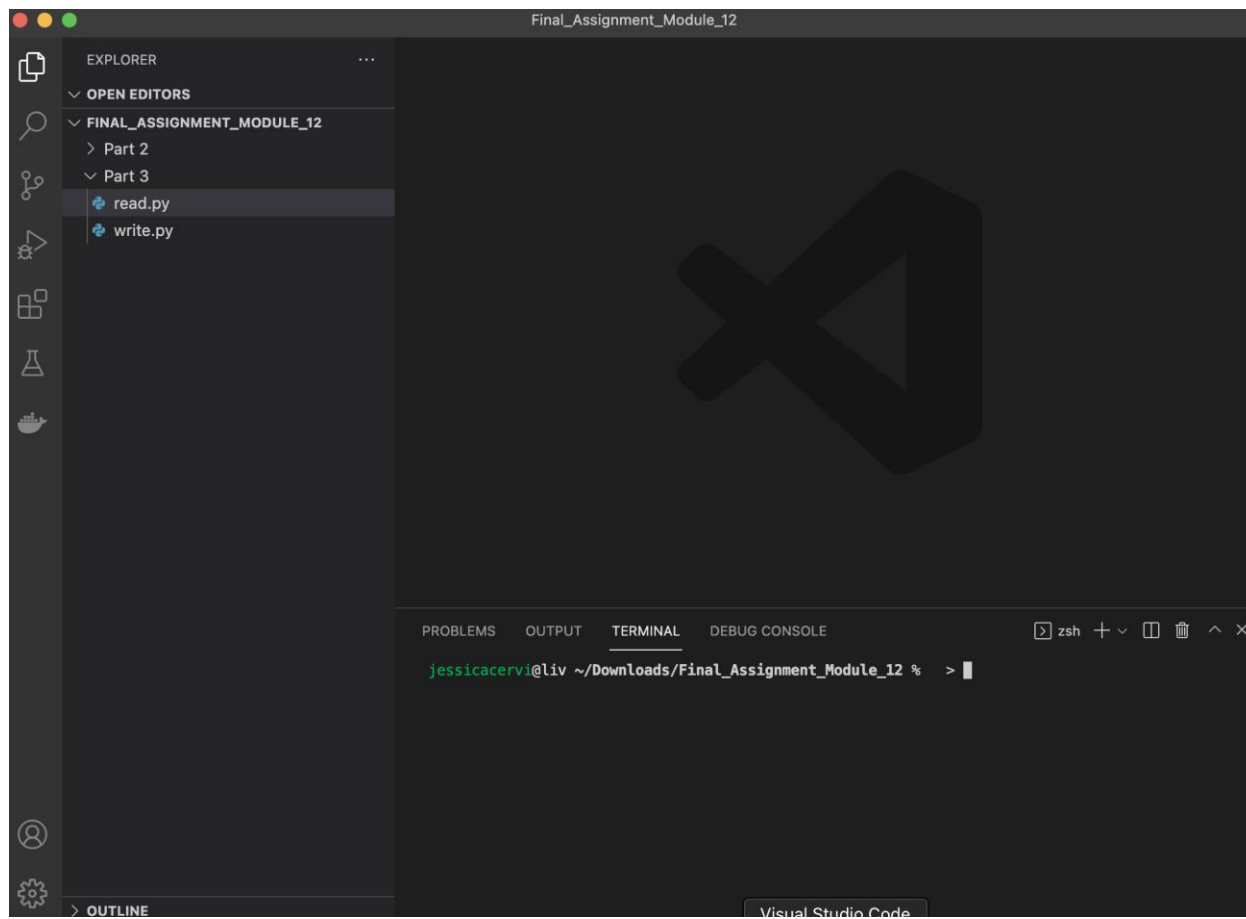
```
bliv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 create.py
bliv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 insert.py
bliv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 insert.py
bliv ~/Downloads/Final_Assignment_Module_12/Part 1 % > python3 selects.py
('29f95d19-cd50-4d81-9f7a-696bbacd4d9e', '2021-10-27 11:29:43')
('86447997-8bbf-40c4-8706-5e84d576290b', '2021-10-27 11:29:44')
('142e51fe-af20-43b2-b751-c109268ab25e', '2021-10-27 11:29:45')
```

Part 2d)



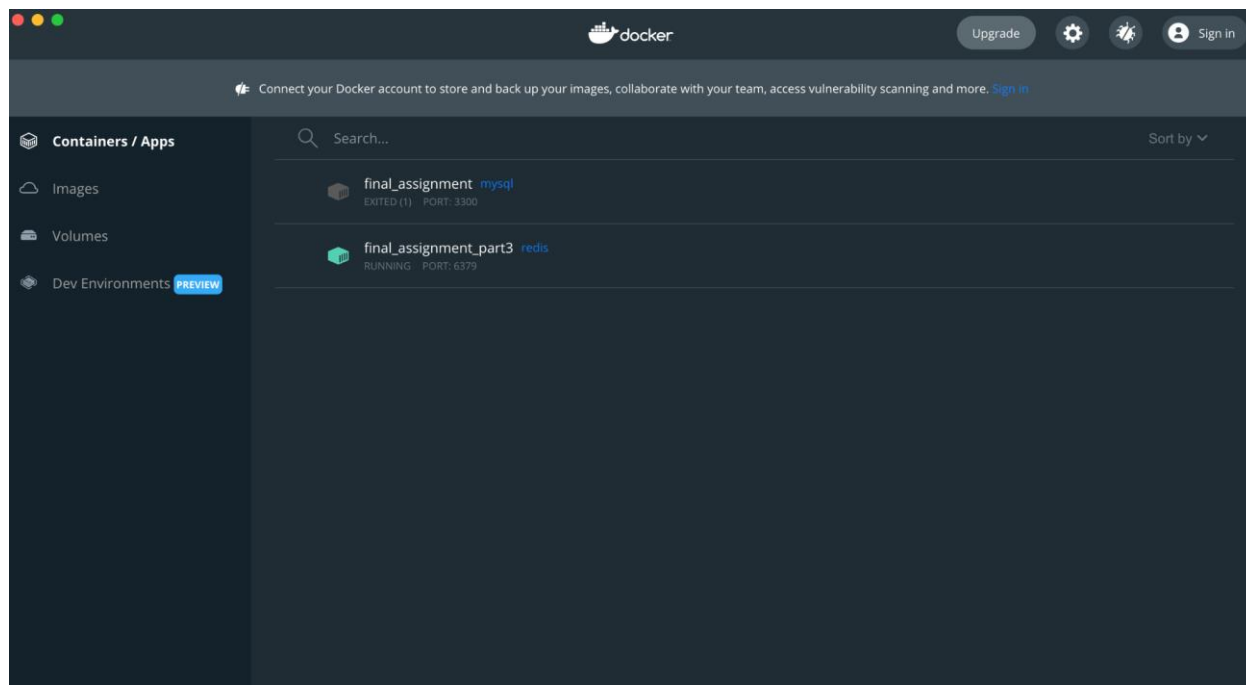
Part 3

Part 3a)

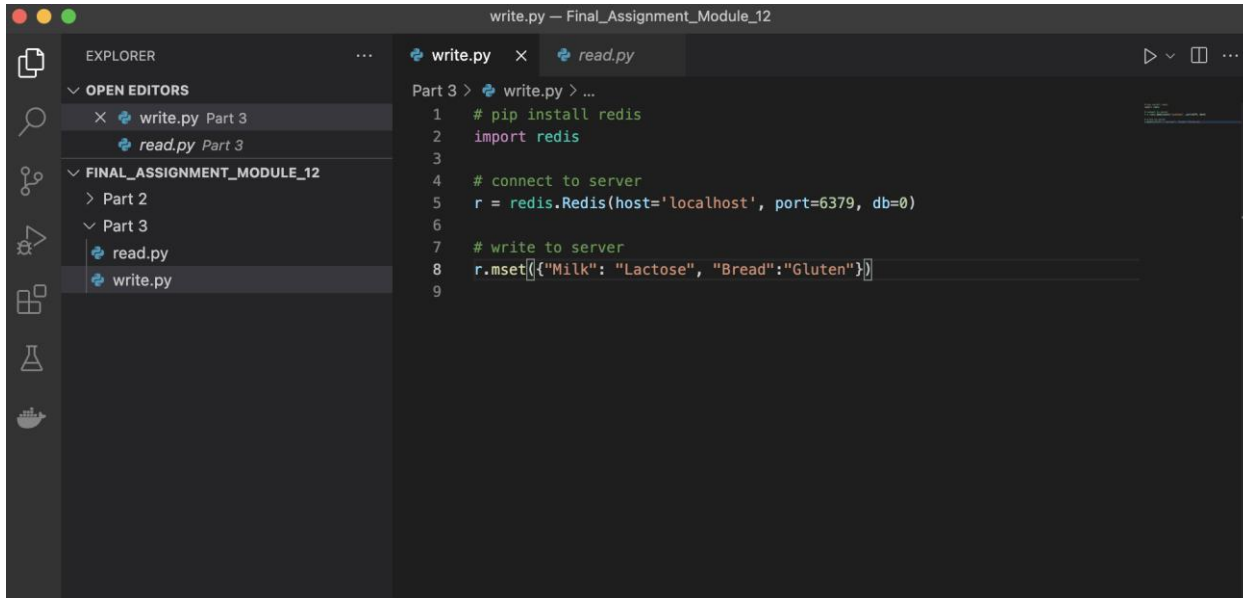


Part 3b)

```
[ [redacted]@liv ~ % > docker run -p 6379:6379 --name final_assignment_part3 ]  
-d redis  
c2406eceb474e16b67d5b826267684a1fcde067b90f755b97fe1269f02ef6d4e  
[redacted]@liv ~ % > ]
```



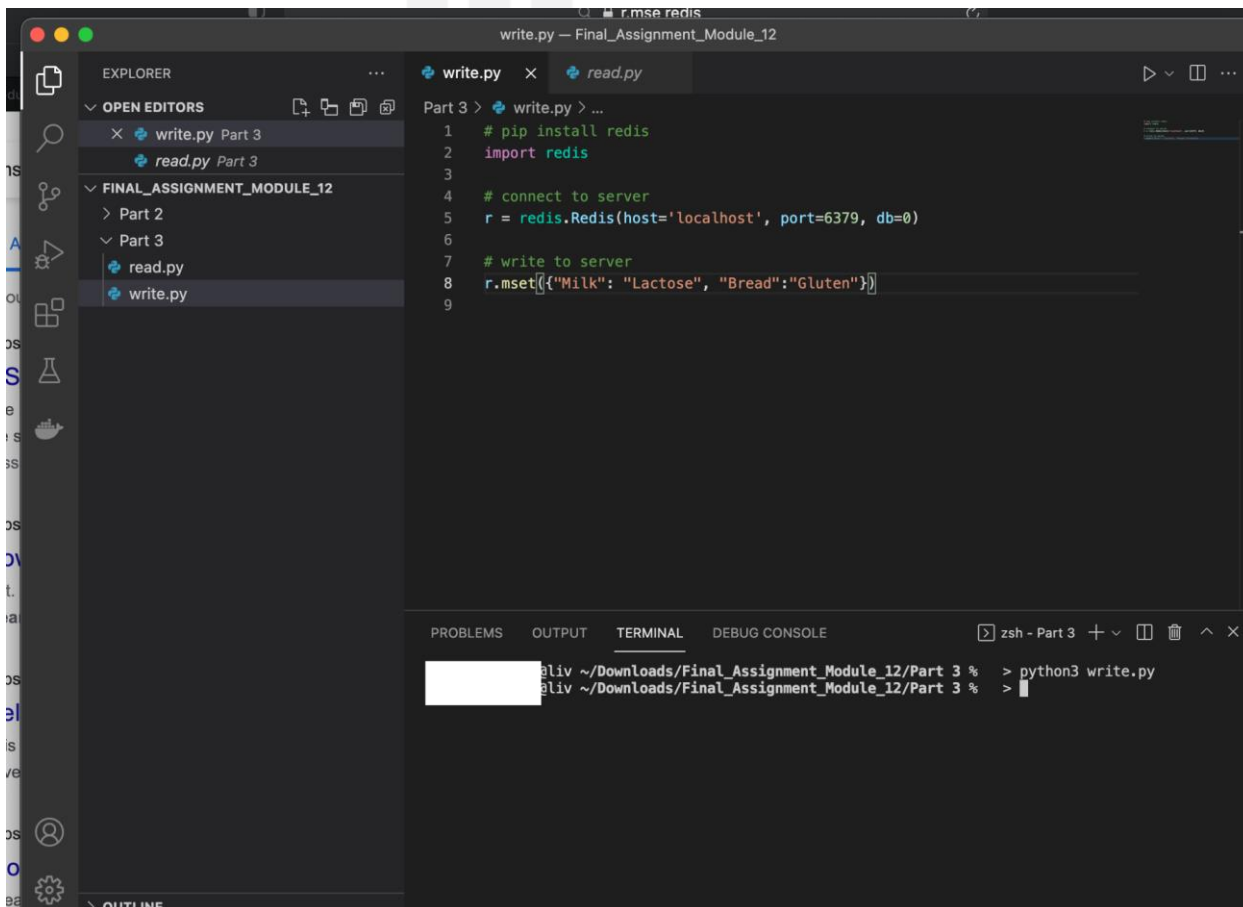
Part 3c)



The screenshot shows the VS Code editor interface. The Explorer panel on the left shows the project structure: **FINAL_ASSIGNMENT_MODULE_12** > **Part 3** > **write.py**. The main editor window displays the code in **write.py**:

```
1 # pip install redis
2 import redis
3
4 # connect to server
5 r = redis.Redis(host='localhost', port=6379, db=0)
6
7 # write to server
8 r.mset({"Milk": "Lactose", "Bread": "Gluten"})
9
```

Part 3d)



The screenshot shows the VS Code editor interface with a terminal window open at the bottom. The Explorer panel on the left shows the project structure: **FINAL_ASSIGNMENT_MODULE_12** > **Part 3** > **write.py**. The main editor window displays the code in **write.py**:

```
1 # pip install redis
2 import redis
3
4 # connect to server
5 r = redis.Redis(host='localhost', port=6379, db=0)
6
7 # write to server
8 r.mset({"Milk": "Lactose", "Bread": "Gluten"})
9
```

The terminal window shows the command prompt and the execution of the script:

```
aliv ~/Downloads/Final_Assignment_Module_12/Part 3 % > python3 write.py
aliv ~/Downloads/Final_Assignment_Module_12/Part 3 % >
```

Part 3e)

```
Part 3 > read.py > ...
1 # pip install redis
2 import redis
3
4 # connect to server
5 r = redis.Redis(host='localhost', port=6379, db=0)
6
7 # read items
8 print(r.get("Milk"))
9 print([r.get("Bread")])
```

Part 3f)

```
Part 3 > read.py > ...
1 # pip install redis
2 import redis
3
4 # connect to server
5 r = redis.Redis(host='localhost', port=6379, db=0)
6
7 # read items
8 print(r.get("Milk"))
9 print([r.get("Bread")])
```

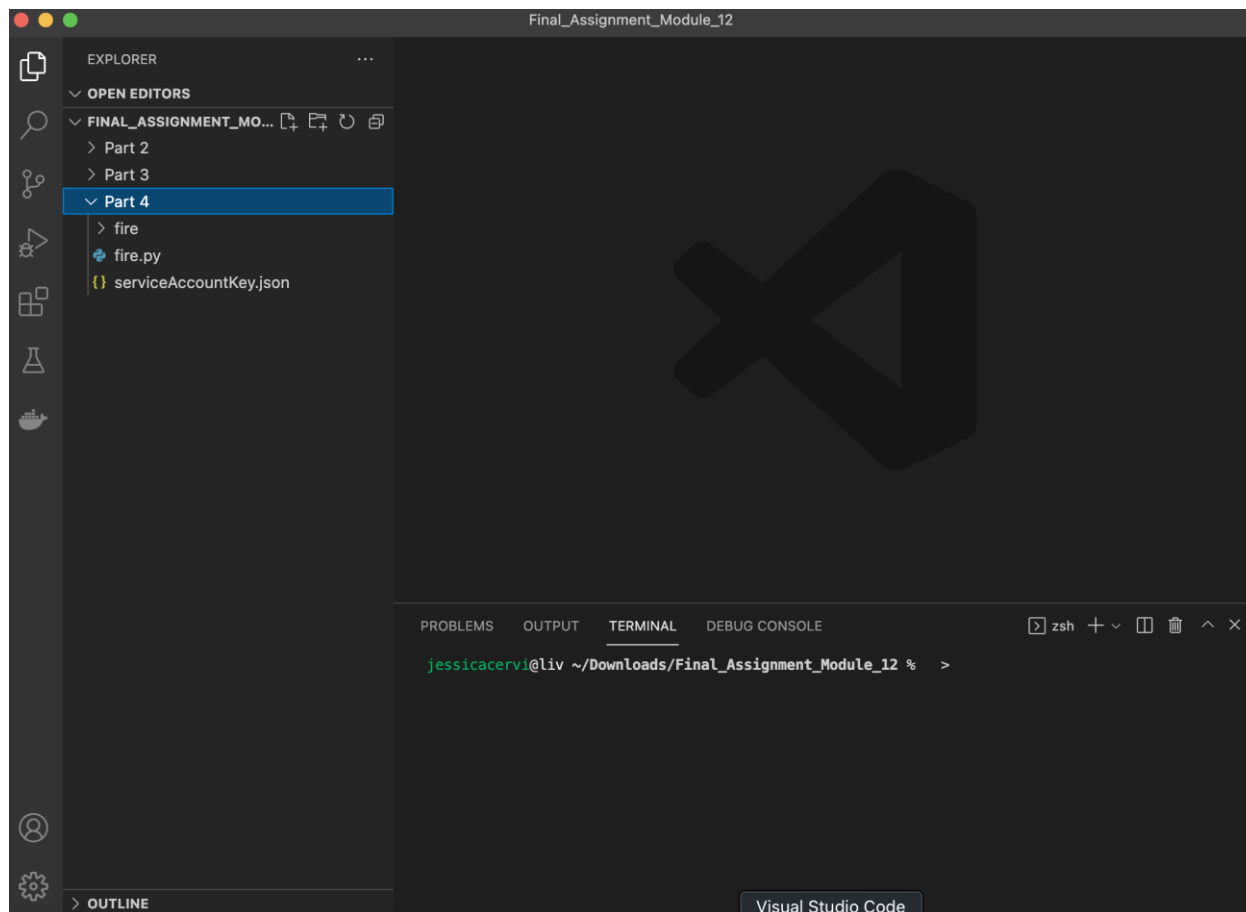
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

zsh - Part 3

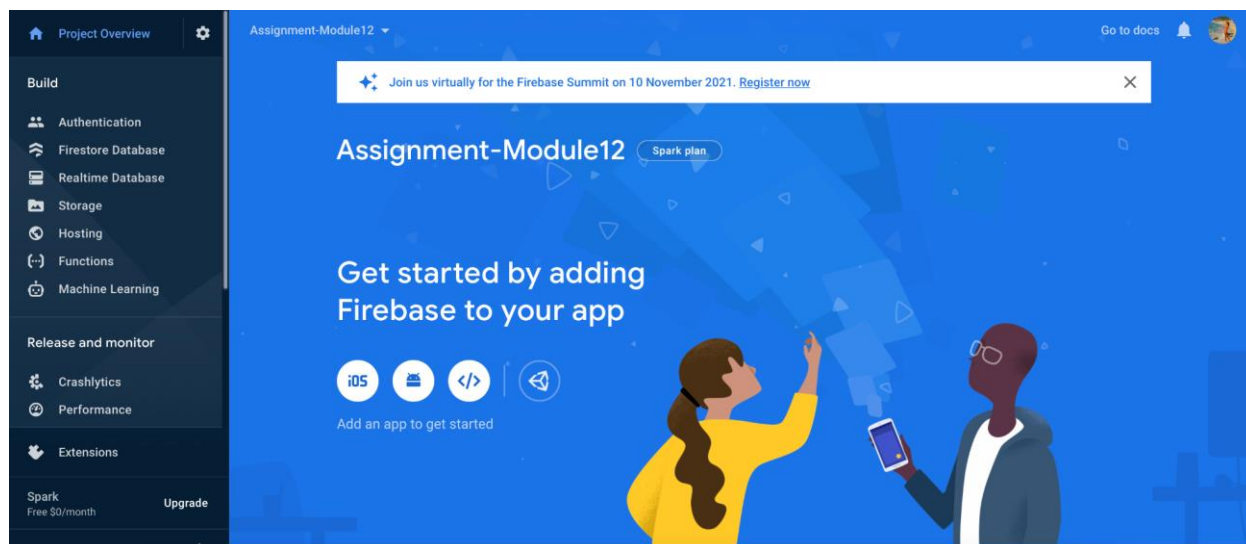
```
@liv ~/Downloads/Final_Assignment_Module_12/Part 3 % > python3 write.py
@liv ~/Downloads/Final_Assignment_Module_12/Part 3 % > python read.py
Lactose
Gluten
@liv ~/Downloads/Final_Assignment_Module_12/Part 3 % >
```

Part 4

Part 4a)



Part 4b)



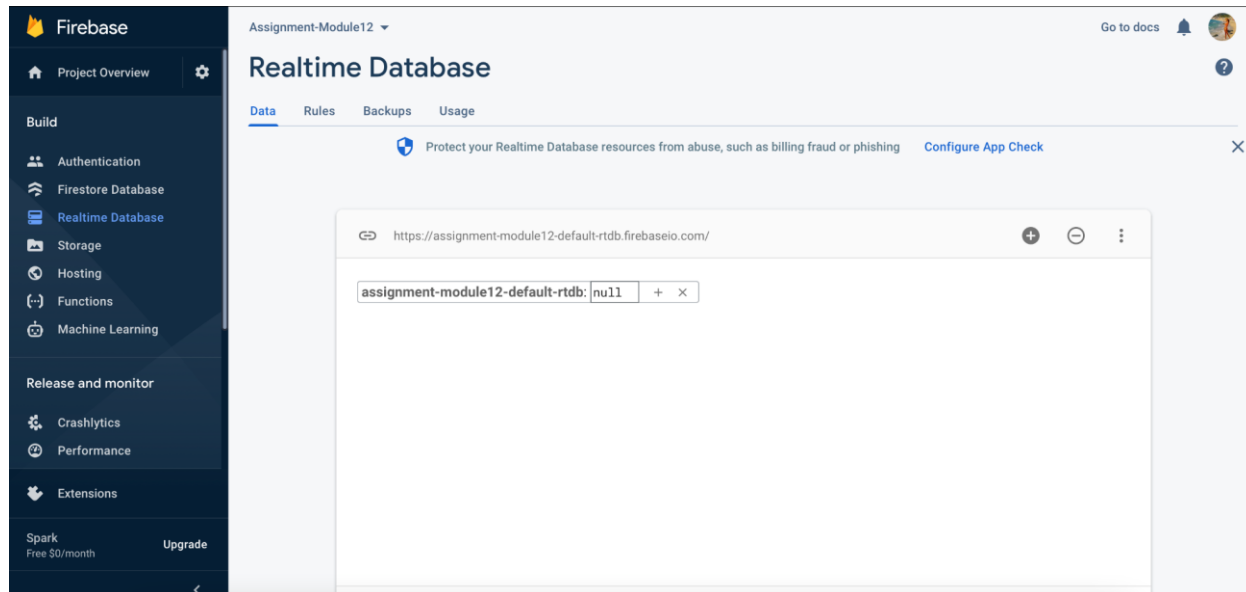
Part 4c)

The screenshot shows the Firebase Project settings page for 'Assignment Module 12'. The 'Service accounts' tab is selected. On the left sidebar, the 'Build' section is expanded, showing options like Authentication, Firestore Database, Realtime Database, Storage, Hosting, Functions, and Machine Learning. The 'Release and monitor' section is also visible. The main content area shows the 'Firebase Admin SDK' configuration. It includes a section for 'Legacy credentials' with 'Database secrets' and 'All service accounts' (3 service accounts). The 'Firebase Admin SDK' section provides instructions on using the service account to authenticate multiple Firebase features. It lists the 'Firebase service account' as 'firebase-adminsdk-25n8q@assignment-module12.iam.gserviceaccount.com'. Below this, there is an 'Admin SDK configuration snippet' with radio buttons for Node.js, Java, Python (selected), and Go. The Python snippet shows the code to initialize the Firebase Admin SDK using the service account key.

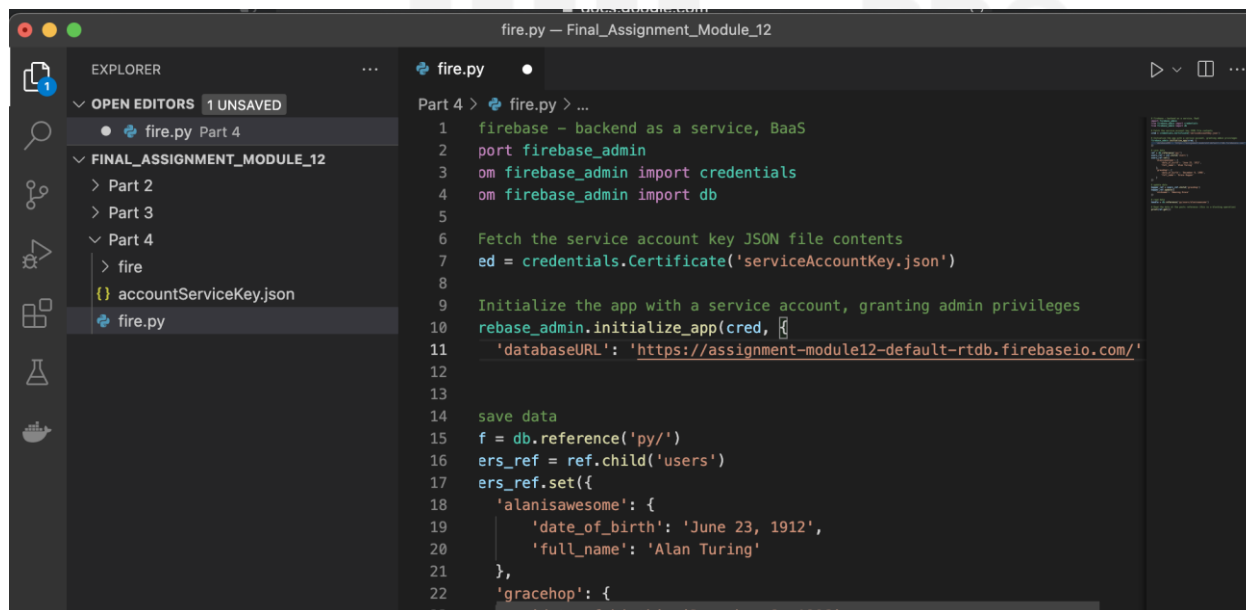
The screenshot shows a code editor with the file 'accountServiceKey.json' open. The file contains the following JSON content:

```
1 {
2   "type": "service_account",
3   "project_id": "assignment-module12",
4   "private_key_id": " ",
5   "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBgkqhkiG9w0BAQEF
6   "client_email": "firebase-adminsdk-25n8q@assignment-module12.iam.gservice
7   "client_id": " ",
8   "auth_uri": "https://accounts.google.com/o/oauth2/auth",
9   "token_uri": "https://oauth2.googleapis.com/token",
10  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/cert
11  "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x50
12
13 }
```

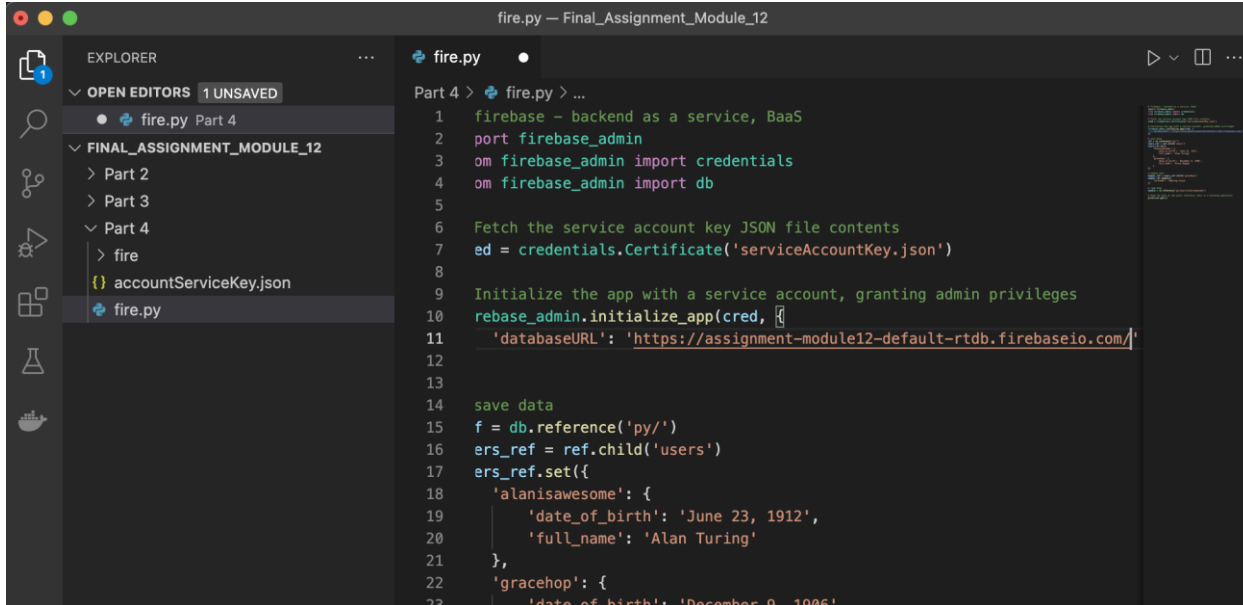
Step 4d)



Step 4e)

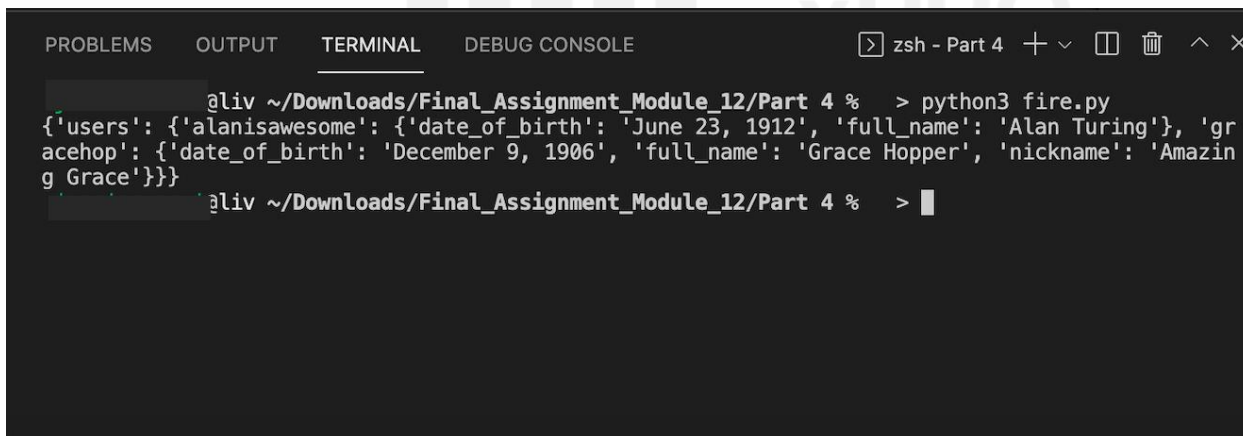


Step 4f)



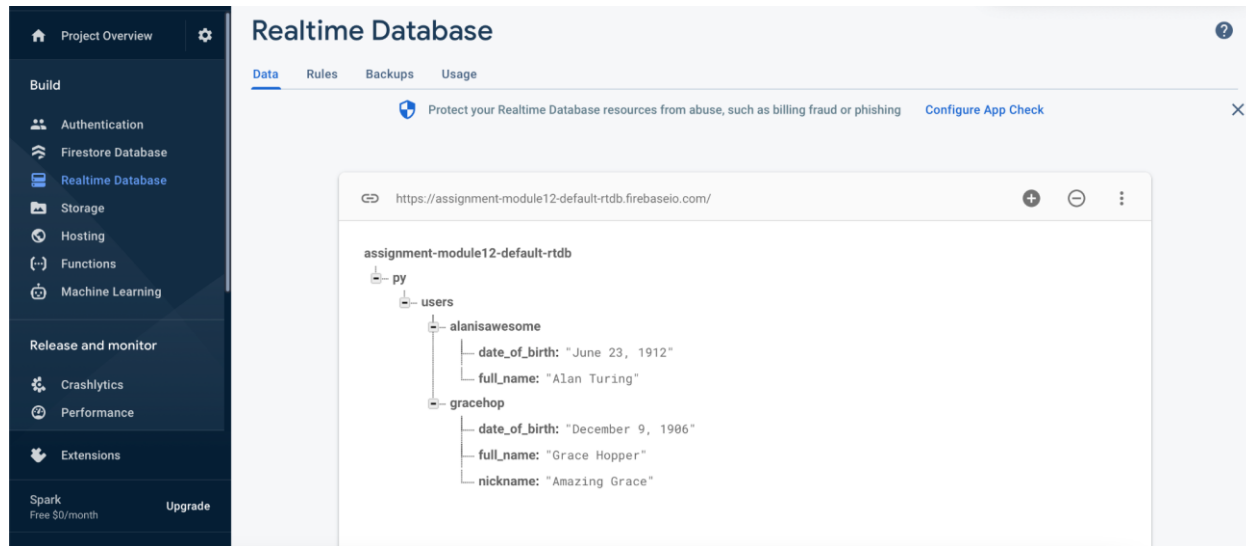
```
Part 4 > fire.py > ...
1  firebase - backend as a service, BaaS
2  port firebase_admin
3  om firebase_admin import credentials
4  om firebase_admin import db
5
6  Fetch the service account key JSON file contents
7  ed = credentials.Certificate('serviceAccountKey.json')
8
9  Initialize the app with a service account, granting admin privileges
10 rebase_admin.initialize_app(cred, {
11     'databaseURL': 'https://assignment-module12-default-rtdb.firebaseio.com/'
12 })
13
14 save data
15 f = db.reference('py/')
16 ers_ref = ref.child('users')
17 ers_ref.set({
18     'alanisawesome': {
19         'date_of_birth': 'June 23, 1912',
20         'full_name': 'Alan Turing'
21     },
22     'gracehop': {
23         'date_of_birth': 'December 9, 1906'
```

Step 4g)



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE zsh - Part 4 + - - - - -
@liv ~/Downloads/Final_Assignment_Module_12/Part 4 % > python3 fire.py
{'users': {'alanisawesome': {'date_of_birth': 'June 23, 1912', 'full_name': 'Alan Turing'}, 'gracehop': {'date_of_birth': 'December 9, 1906', 'full_name': 'Grace Hopper', 'nickname': 'Amazing Grace'}}}
@liv ~/Downloads/Final_Assignment_Module_12/Part 4 % > |
```

Step 4h)



The screenshot shows the Google Cloud Realtime Database console. The left sidebar contains a navigation menu with options: Project Overview, Build (Authentication, Firestore Database, Realtime Database, Storage, Hosting, Functions, Machine Learning), Release and monitor (Crashlytics, Performance), Extensions, and Spark (Free \$0/month, Upgrade). The main panel is titled 'Realtime Database' and has tabs for Data, Rules, Backups, and Usage. A notification bar at the top states: 'Protect your Realtime Database resources from abuse, such as billing fraud or phishing. Configure App Check'. The 'Data' tab is active, showing a tree view of the database structure for 'assignment-module12-default-rtdb'. The structure is as follows:

```

assignment-module12-default-rtdb
├── py
│   └── users
│       ├── alanisawesome
│       │   ├── date_of_birth: "June 23, 1912"
│       │   └── full_name: "Alan Turing"
│       └── gracehop
│           ├── date_of_birth: "December 9, 1906"
│           ├── full_name: "Grace Hopper"
│           └── nickname: "Amazing Grace"

```

